



MANUAL TÉCNICO

Sistema Integral de Gestión para la UICM

Instalación, configuración, tareas programadas y despliegue a
producción

Versión 1.0 · Julio 2026

Contenido

1. Introducción

2. Arquitectura y stack tecnológico

3. Requisitos del sistema

4. Instalación local

- Clonar y preparar el proyecto

- Base de datos

- Compilar assets

5. Variables de entorno

6. Integraciones externas (MercadoPago y Correo)

7. Comandos artisan y tareas programadas

8. Despliegue a producción

9. Seguridad

10. Solución de problemas técnicos

1. Introducción

Este manual documenta el **Sistema Integral de Gestión para la UICM** desde el lado técnico: cómo está construido, cómo instalarlo en un entorno local, cómo configurarlo y cómo mantenerlo funcionando en producción.

Está dirigido a quien instale, despliegue o dé mantenimiento al sistema — no cubre el uso funcional de cada pantalla, eso está en los manuales de usuario por rol (Administrativo, Alumno, Profesor, Aspirante). Para entender la lógica de negocio a fondo (entidades, flujos, reglas), consulta también `mapa-sistema.html` en la carpeta de referencia interna del repositorio.

 Este documento asume familiaridad básica con la terminal, Git y conceptos generales de un proyecto Laravel.

2. Arquitectura y stack tecnológico

COMPONENTE	DETALLE
Backend	Laravel 12 (PHP 8.2+)
Base de datos	SQLite (archivo único, sin servidor de BD separado)
Frontend	Blade Templates + Tailwind CSS 3 + Alpine.js 3, empaquetado con Vite
Pagos	MercadoPago (Checkout Pro), vía SDK oficial <code>mercadopago/dx-php</code>
Correo	SMTP, configurable en tiempo de ejecución desde el panel de administración
Colas / caché / sesiones	Driver <code>database</code> (misma base SQLite, sin Redis)

La arquitectura sigue el patrón estándar de Laravel (MVC): controladores en `app/Http/Controllers`, modelos Eloquent en `app/Models`, vistas Blade en `resources/views`. No hay una API separada ni un frontend desacoplado — todo se sirve como una aplicación monolítica tradicional.



Usar SQLite en vez de MySQL/PostgreSQL fue una decisión deliberada para mantener el despliegue simple: no requiere un servidor de base de datos aparte, solo un archivo dentro del proyecto.

3. Requisitos del sistema

Requisito	Versión mínima
PHP	8.2 o superior, con extensiones <code>pdo_sqlite</code> , <code>mbstring</code> , <code>fileinfo</code> , <code>gd</code> o <code>imagick</code>
Composer	2.x
Node.js	20 o superior (requerido por Vite 7)
npm	Incluido con Node.js
Servidor web	Apache o Nginx con soporte para reescritura de URLs (o <code>php artisan serve</code> en local)

 No se requiere MySQL, PostgreSQL, Redis ni un servidor de colas aparte — el proyecto usa SQLite y drivers basados en base de datos para todo. Instalar de más solo añade complejidad innecesaria.

4. Instalación local

4a Clonar y preparar el proyecto

```
# Clonar el repositorio
git clone <url-del-repositorio> uicm
cd uicm

# Instalar dependencias de PHP
composer install

# Copiar el archivo de entorno de ejemplo
cp .env.example .env

# Generar la clave de la aplicación (APP_KEY)
php artisan key:generate
```

! `php artisan key:generate` se corre **una sola vez**, idealmente antes de capturar cualquier credencial en el sistema. Ver la sección 9 (Seguridad) — regenerar el `APP_KEY` después de configurar MercadoPago o Correo invalida esas credenciales cifradas.

4b Base de datos

```
# Crear el archivo de base de datos SQLite (si no existe)
touch database/database.sqlite

# Correr las migraciones (crea todas las tablas)
php artisan migrate

# Opcional: cargar datos de ejemplo
php artisan db:seed
```

Confirma que en tu `.env` tengas `DB_CONNECTION=sqlite` y que la ruta del archivo apunte correctamente (Laravel usa `database/database.sqlite` por defecto cuando no se especifica `DB_DATABASE`).

4c

Compilar assets

```
# Instalar dependencias de Node
npm install

# Compilar CSS/JS para producción
npm run build

# – o, en desarrollo, dejar Vite observando cambios –
npm run dev
```

i Cada vez que se agreguen clases nuevas de Tailwind o se edite un archivo JS/CSS fuente, hay que volver a correr `npm run build` (o tener `npm run dev` corriendo) para que el cambio se vea reflejado — Blade no compila los assets por sí solo.

```
# Enlace simbólico para servir archivos subidos (carrusel, documentos, etc.)
# Se corre UNA SOLA VEZ al configurar el proyecto (local o producción)
php artisan storage:link

# Levantar el servidor de desarrollo
php artisan serve
```

5. Variables de entorno

Las variables más relevantes del `.env` :

Variable	Uso
<code>APP_ENV</code>	<code>local</code> en desarrollo, <code>production</code> en el servidor real.
<code>APP_DEBUG</code>	<code>true</code> en local (muestra errores detallados), debe ser <code>false</code> en producción por seguridad.
<code>APP_URL</code>	Dominio base de la instalación — debe reflejar la URL real (local, ngrok o el dominio de producción) para que los enlaces generados por el sistema sean correctos.
<code>APP_KEY</code>	Clave de cifrado de la aplicación. Ver advertencia en la sección 4a y 9.
<code>DB_CONNECTION</code>	<code>sqlite</code> — no se cambia salvo que se migre a otro motor de base de datos.
<code>MAIL_*</code>	Valores de referencia/fallback. La configuración de correo realmente usada se gestiona desde el panel <code>Admin → APIs</code> (pestaña Correo), guardada en base de datos, no aquí.

 Las credenciales de **MercadoPago** y de **Correo** no van en el `.env` en este proyecto — se configuran desde el panel `Admin → APIs` ya con el sistema corriendo, y se guardan cifradas en la base de datos. Ver la siguiente sección.

6. Integraciones externas (MercadoPago y Correo)

A diferencia de un proyecto Laravel típico, este sistema **no lee las credenciales de servicios externos del `.env`** — se administran en tiempo de ejecución desde `/admin/apis`, con rol `admin`, para que se puedan cambiar sin tocar código ni volver a desplegar.

- **Pestaña MercadoPago:** `access_token`, `public_key` y modo sandbox/producción. El producto usado es **Checkout Pro**.
- **Pestaña Correo:** host, puerto, usuario y contraseña SMTP (o driver `sendmail` si el hosting no requiere credenciales), además del remitente y el dominio institucional usado para generar los correos de alumnos (`matricula@dominio`).

! `access_token` de MercadoPago y la contraseña SMTP se guardan **cifrados con AES-256** (cast `encrypted` de Laravel) usando el `APP_KEY` del `.env`. Si el `APP_KEY` se regenera **después** de capturar esas credenciales, quedan indescifrables y hay que volver a escribirlas desde el panel. Genera el `APP_KEY` definitivo **antes** de configurar estas dos pestañas.

i Al pasar de sandbox a producción con MercadoPago: solo se cambian las credenciales en `/admin/apis` (usar las que empiezan con `APP_USR-`, no `TEST-`) y se actualiza la URL del sitio en el panel de desarrolladores de MercadoPago. No requiere cambios de código.

7. Comandos artisan y tareas programadas

El sistema depende de tareas automáticas para funcionar correctamente día a día. Todas están definidas en `routes/console.php` y se ejecutan mediante el **scheduler** de Laravel:

Hora	Comando	Qué hace
00:05 diario	<code>periodos:sincronizar</code>	Abre los cuatrimestres cuya fecha de inicio ya llegó y cierra los que ya vencieron (solo los que tienen el modo "automático" activado).
09:00 AM diario	<code>uicm:resumen-diario</code>	Envía un correo de resumen a cada usuario administrativo: aspirantes pendientes, pagos pendientes, alumnos sin grupo, etc.
01:00 AM, día 1 de cada mes	<code>pagos:generar-colegiatura</code>	Genera el cargo de colegiatura del mes para cada alumno activo (idempotente: no duplica si ya existe el cargo del mes).
01:30 AM diario	<code>pagos:generar-reinscripcion</code>	Genera el cargo de reinscripción para alumnos que deben avanzar de cuatrimestre.

7a Probar el scheduler en local

```
# Deja el scheduler corriendo en una terminal aparte;  
# ejecuta cada tarea programada en su horario real, sin esperar al cron del sistema operativo  
php artisan schedule:work
```

7b

Ejecutar un comando manualmente

```
# Útil para demos o para forzar la ejecución sin esperar al horario programado
php artisan uicm:resumen-diario
php artisan periodos:sincronizar
php artisan pagos:generar-colegiatura
php artisan pagos:generar-reinscripcion
```

8. Despliegue a producción

Checklist para pasar de un entorno local (con ngrok) a un hosting real (ej. Hostinger/cPanel):

8a Variables en `.env`

```
APP_URL=https://tudominio.com ← dominio real de producción
APP_ENV=production
APP_DEBUG=false
```

8b Configurar MercadoPago y Correo

Desde `/admin/apis` (ver sección 6): cambiar a credenciales de producción de MercadoPago (`APP_USR-`) y a los datos SMTP reales del dominio (o `sendmail` si el hosting lo soporta sin credenciales). Actualizar también las `back_urls` /webhook de MercadoPago al dominio real.

8c Cron job (una sola vez)

En el panel de hosting (ej. Hostinger → Cron Jobs), agregar una única entrada que active **todo** el scheduler:

```
* * * * * php /ruta-real-del-proyecto/artisan schedule:run >> /dev/null 2>&1
```

! Sin este cron job, los cuatrimestres no abren/cierran solos y los correos automáticos no se envían — nada del scheduler funciona sin él.

8d Enlace de almacenamiento

```
php artisan storage:link
```

Necesario para que las imágenes del carrusel y otros archivos subidos desde el panel de administración se sirvan correctamente.

8e

Revisar `trustProxies`

En `bootstrap/app.php` el proyecto trae `$middleware->trustProxies(at: '*')`, necesario en desarrollo porque el tráfico pasa por el proxy de ngrok (para que Laravel detecte bien HTTPS al generar URLs para MercadoPago).

- ❗ Confiar en `'*'` en producción real es un riesgo si el hosting no coloca el sitio detrás de un único proxy de confianza (alguien podría falsificar su IP/protocolo con encabezados `X-Forwarded-*`). Al desplegar a un hosting real, cambiar `at: '*'` por la IP real del proxy del hosting, o quitar `trustProxies` si no hay un proxy intermedio.

9. Seguridad

Medida	Detalle
Rate limiting en login	Bloqueo progresivo por IP: 1 min → 5 min → 15 min → 1 hora tras intentos fallidos repetidos. Mensajes de error genéricos (no revelan si falló el correo o la contraseña). Tabla <code>login_attempts</code> .
Credenciales cifradas	<code>access_token</code> de MercadoPago y contraseña SMTP cifrados con AES-256 usando <code>APP_KEY</code> (ver sección 6).
Control de acceso a archivos	Ruta <code>/admin/archivo/{path}</code> valida al dueño antes de servir cualquier documento. El personal administrativo puede ver cualquier expediente (parte de su trabajo); un alumno solo puede ver sus propios archivos.
Aislamiento por rol	Middleware <code>rol:X</code> en todas las rutas protegidas, con <code>abort_if</code> en los controladores como segunda capa de verificación.
<code>trustProxies</code>	Revisar antes de producción real (ver sección 8e).
Consultas a base de datos	Se usa Eloquent y el Query Builder de Laravel en todo el proyecto, que parametrizan las consultas por defecto — sin hallazgos de inyección SQL en auditoría.

10. Solución de problemas técnicos

Problema	Qué hacer
Los estilos nuevos de Tailwind no se ven	Corre <code>npm run build</code> (o ten <code>npm run dev</code> activo). Blade no compila los assets automáticamente.
Las imágenes subidas no se muestran (404)	Falta el enlace simbólico: corre <code>php artisan storage:link</code> .
Error "could not decrypt" en credenciales de MercadoPago/Correo	El <code>APP_KEY</code> se regeneró después de capturar esas credenciales. Vuelve a escribirlas desde <code>/admin/apis</code> con el <code>APP_KEY</code> actual.
Los periodos no abren/cierran solos, o no llegan los correos automáticos	Falta el cron job de producción, o el servidor no lo está ejecutando. Verifica la configuración en el panel de hosting (sección 8c).
Los enlaces de retorno de MercadoPago salen en <code>http://</code> en vez de <code>https://</code>	Revisa <code>APP_URL</code> en <code>.env</code> y la configuración de <code>trustProxies</code> en <code>bootstrap/app.php</code> .
Error de base de datos al iniciar	Verifica que exista <code>database/database.sqlite</code> y que el proceso PHP tenga permisos de escritura sobre ese archivo y su carpeta.
Error 500 sin detalle en producción	Revisa <code>storage/logs/laravel.log</code> . Si <code>APP_DEBUG=false</code> (correcto en producción), el navegador no mostrará el detalle del error — solo el log.

i Para cualquier duda sobre la lógica de negocio (qué hace cada módulo, cómo se relacionan las entidades), consulta `docs/referencia-interna/mapa-sistema.html` — este manual técnico se enfoca en instalación, configuración y mantenimiento, no en el funcionamiento interno de cada módulo.